1. Abstract

This project is a console-based Random Maze Generator and Solver written in the C programming language. The program allows the user to generate a random maze of any chosen size (up to 99×99), automatically display it, attempt to solve it using recursive backtracking, and save or load mazes to and from text files. The maze is generated using a recursive backtracking carving algorithm and stored internally as a global 2D character array. The project demonstrates the practical application of fundamental programming concepts such as functions, recursion, arrays, control flow, and file handling — all of which are part of the CSE115 syllabus.

2. Introduction

Objective of the Project

The objective of this project is to design and implement a fully working maze generator and solver in C, using only the topics taught in CSE115. The program should:

1. Generate a random maze of user-defined dimensions.
2. Display the maze on the console using ASCII characters.
3. Solve the maze automatically using recursion and show the path.
4. Save the maze to a text file and load it back later.
5. Provide a clean menu-driven interface.

Background and Significance

Maze generation and solving are classic problems in computer science. They serve as a strong foundation for learning core ideas such as 2D arrays, recursion, backtracking, and file I/O — concepts every introductory programming student must master.

Building this project from scratch helped the team move beyond simple textbook exercises and apply many CSE115 topics together inside a single, real-world style program. It also reinforced disciplined use of pointers and structures for organising data.

3. Features and Implementation

The project provides the following features. Each feature is implemented as a separate function in maze.c.

3.1 Menu-Driven Interface

A do…while loop combined with a switch statement repeatedly displays a menu and processes the user's choice until they choose to exit.

```
do {
    printf("=== Random Maze Generator & Solver ===\n...");
```

```
    printf("Enter your choice: ");
    scanf("%d", &choice);
    switch (choice) {
        case 1: /* Generate */ break;
        case 2: /* Solve    */ break;
        case 3: /* Save     */ break;
        case 4: /* Load     */ break;
        case 5: printf("Exiting...\n"); break;
        default: printf("Invalid choice.\n");
    }
} while (choice != 5);
```

3.2 Random Maze Generation

The maze is generated using a recursive backtracking carving algorithm. The grid is first filled entirely with walls (#); then a recursive function carve() starts from cell (1,1) and explores in four directions in randomised order (using a Fisher–Yates shuffle of an index array). For each direction it jumps two cells, and if the new cell is still a wall it knocks down the wall in between and recurses. The user-supplied dimensions are rounded down to the nearest odd values so that the carving lattice fits cleanly. After carving, S (start) is placed at (1,1) and E (end) at (R-2, C-2), guaranteeing that every maze produced is fully connected and solvable.

```
void carve(int r, int c) {
    int d[4] = {0, 1, 2, 3};
    int dr[4] = {-2, 2, 0, 0}, dc[4] = {0, 0, -2, 2};
    m[r][c] = ' ';
    /* Fisher-Yates shuffle of direction order */
    for (i = 3; i > 0; i--) {
        j = rand() % (i + 1); /* swap d[i] and d[j] */
    }
    for (i = 0; i < 4; i++) {
        nr = r + dr[d[i]]; nc = c + dc[d[i]];
        if (nr > 0 && nr < R-1 && nc > 0 && nc < C-1 && m[nr][nc] == '#') {
            m[r+dr[d[i]]/2][c+dc[d[i]]/2] = ' '; carve(nr, nc); } } }
```

3.3 Auto-Display After Generation / Load

When a maze is generated or loaded from a file, it is shown on the console immediately so that the user does not need a separate ‘display’ menu option. The show() function uses ANSI escape codes to colour-code the output: walls (#) in blue, the start cell (S) on a green background, the end cell (E) on a red background, and the solved path (*) in yellow.

3.4 Recursive Maze Solver

The solve() function uses recursion with backtracking to explore all four directions from the start cell until it reaches E. As the recursion descends, each visited cell is temporarily marked with '*'; if the recursive call eventually reaches E, the marker is kept (forming the visible path) and the original 'S' is restored at the start cell. If a cell only leads to dead ends, its original character is restored on the way back so it can be reused by other branches. Before each solve attempt, main() runs a small loop that wipes any leftover '*' markings from a previous run, guaranteeing the solver always starts with a clean maze.

```
int solve(int r, int c) {
```

```
    if (r < 0 || c < 0 || r >= R || c >= C) return 0;  /* bounds */
    if (m[r][c] == '#' || m[r][c] == '*') return 0;
    if (m[r][c] == 'E') return 1;              /* base case */

    char old = m[r][c];
    m[r][c] = '*';

    if (solve(r+1, c) || solve(r, c+1) ||
        solve(r-1, c) || solve(r, c-1)) {
        if (old == 'S') m[r][c] = 'S';   /* keep '*' otherwise */
        return 1;
    }
    m[r][c] = old;   /* backtrack: erase dead-end marker */
    return 0;
}
```

3.5 Save Maze to File

The current maze is written to a plain-text file using fopen and fprintf. The user is prompted for a filename; the first line of the file stores the size, followed by the grid contents row by row.

```
fprintf(fp, "%d %d\n", R, C);
for (i = 0; i < R; i++) {
    for (j = 0; j < C; j++) fprintf(fp, "%c", m[i][j]);
    fprintf(fp, "\n");
}
```

3.6 Load Maze from File

A previously saved maze is read back using fopen and fscanf. The user is prompted for the filename, and after a successful load the maze is automatically displayed.

3.7 Use of 2D Arrays and Global State

All maze data is stored in a single global 2D character array m[101][101], with two global integers R and C holding the current row and column counts. Because the maze is shared by every helper function (carve, gen, show, solve, saveMaze, loadMaze), keeping it as a global avoids passing the array around between calls and makes the recursion in carve() and solve() simpler to read.

```
/* Globals shared by every helper function */
int R, C;                /* current dimensions */
char m[101][101];        /* the maze grid */
/* Used directly inside carve, gen, show, solve, saveMaze, loadMaze */
```

4. Member Contribution

Feature / Task	Implemented By	Description
Project setup, menu system, main() flow	Khalid Been Hossain (2611404042)	Designed the menu loop, input handling, and overall program structure
Maze generation (gen, carve) and 2D-array layout	Sadeed Ahmad Chowdhury (2613981642)	Implemented recursive backtracking carving, start/end placement, and the global maze layout
Recursive solver (solve) and path-clearing logic	Wamiq Ahnaaf Hasan (2611171042)	Implemented backtracking-based maze solving and path-marking logic
File I/O (saveMaze, loadMaze) and display function	Md. Nasirul Islam (2612716042)	Implemented saving/loading mazes from text files and console display

Note: Adjust this table to reflect the actual contribution of each member.

5. Testing and Results

Test Cases

#	Input / Action	Expected Output	Result	Status
1	Choose option 1 → enter rows=10, cols=10	Maze generated and displayed (10×10 with # borders, S at top-left, E at bottom-right)	☑ Pass	
2	Choose option 1 → enter rows=10, cols=10 (even values)	Maze generated and displayed at the next-lower odd dimensions (9×9), to fit the carving lattice	☑ Pass	
3	Generate a maze, then choose option 2 (Solve)	Maze redisplayed with * marking the path from S to E	☑ Pass	
4	Choose option 2 before generating any maze	"No path found." (R and C are 0, so solve() returns 0 immediately)	☑ Pass	
5	Generate a maze, save as test.txt, exit, restart, load test.txt	Same maze is restored and auto-displayed	☑ Pass	
6	Load a non-existent file (abc.txt)	"File could not be opened."	☑ Pass	
7	Load a hand-edited maze where the path between S and E has been blocked, then choose option 2	"No path found."	☑ Pass	
8	Choose option 5 (Exit)	"Exiting..." and program terminates cleanly	☑ Pass	

Sample Output

After choosing option 1 (Generate Maze with rows=10, cols=15):

```
=== Random Maze Generator & Solver ===
1. Generate Maze
2. Solve Maze
3. Save Maze
4. Load Maze
5. Exit
Enter your choice: 1
Rows and Columns: 10 15




###############
#S            #
# ### ### ### #
#   # # #     #
### # # # ### #
#     #   #   #
# ### ##### # #
#   #     # # #
### ##### # #E#
###############

Colour key (rendered live by show() using ANSI escape codes): # wall (blue)   S start
(green background)   E end (red background)   * path (yellow)
```

After choosing option 2 (Solve Maze):

```
###############
#S************#
# ###*###*###*#
#   #*# #*****#
### #*# # ### #
#   # #   #   #
# ### ##### # #
#   #     # # #
### ##### # #E#
###############
```

Summary of Results

All planned features were successfully implemented and tested. The program runs reliably for any valid input, handles invalid sizes gracefully, and the recursive solver correctly identifies whether a path exists between S and E.

6. Challenges and Solutions

Challenge 1 — Restricted Toolset

Issue: The project was required to stay strictly within the CSE115 lecture slides. This meant we could not use time.h for seeding rand(), INT_MAX, or advanced library functions for parsing input.

Solution: We relied only on rand() (without an explicit srand seed) and used recursive backtracking carving — which is built entirely from concepts taught in CSE115 (recursion, arrays, control flow,

the modulo operator) — to generate fully-connected mazes. Numeric input is read directly with scanf rather than any helper from a library not on the syllabus.

Challenge 2 — Carving Requires Odd Dimensions

Issue: The carve() function jumps two cells at a time, so it only works correctly when the grid has odd row and column counts. Even dimensions left a stray border row/column that broke the algorithm.

Solution: Inside gen() we silently round the user's input down to the nearest odd value before carving (R = (r % 2 == 0) ? r - 1 : r; and likewise for C). Because recursive backtracking carving always produces a fully-connected maze, every generated maze is guaranteed to be solvable.

Challenge 3 — Marking the Path Without Losing the Start Cell

Issue: While marking visited cells with '*' during recursion, the 'S' cell was being overwritten.

Solution: Inside solve(), we save the original character into a local variable old before marking, and after a successful return we restore 'S' if old == 'S'; otherwise the '*' stays in place to draw the path.

Challenge 4 — Reading the Maze Back Correctly From File

Issue: Mixing fscanf for the size header with character-by-character reading of the grid caused the trailing newline after each line to be picked up as part of the next row.

Solution: We use fscanf with the "%c" specifier inside the inner loop to read each cell directly, and an explicit fscanf(fp, "\n") between rows to consume the trailing newline. This keeps the grid aligned with what saveMaze() wrote and avoids any whitespace-skipping side effects.

7. Conclusion

Summary of the Project

The Random Maze Generator & Solver is a complete, single-file C program that demonstrates how multiple introductory programming concepts can be combined to build a useful interactive application. It covers menu design, randomisation, recursive backtracking (used both for carving and for solving), 2D arrays, and file handling — all key topics from CSE115.

What Was Learned

- How to design a clean menu-driven program using do…while and switch.
- How recursion combined with backtracking can elegantly solve search problems.
- How a global 2D array can simplify sharing data across many helper functions, including recursive ones.
- How to use C's file I/O functions (fopen, fprintf, fscanf, fclose) to persist program state.
- The importance of input validation and graceful error handling in user-facing programs.

Possible Future Improvements

- Validate user input for dimensions (reject values below 5 or above 99) and clamp gracefully instead of trusting the user.
- Allow the user to choose the start and end positions.
- Show the shortest path instead of the first-found path (e.g., using BFS).
- Seed rand() with srand(time(NULL)) so each run produces a fresh maze instead of the default deterministic sequence.
- Add a timer to measure how fast the solver finds the path.

End of Report — Group 5, CSE115